Github link-
https://github.com/sanchitsanyam/Congestion-Uncertainty-Aware-Rider-Dispatch

CUARD

Congestion & Uncertainty-Aware Rider Dispatch

Zomathon Problem Statement 1 · 65%+ Rider Wait Reduction · ₹90+ Crore Annual Savings

45K+	52%	65%	₹90cr
Orders Analyzed	FOR Signal Improvement	Wait Time Reduction	Annual Savings

Presented to: Zomato Engineering · Hackathon Submission

THE PROBLEM: FOR SIGNAL BIAS

😊 Honest	🚴 Rider-Triggered	⏰ Early Marker
Marks ready when food is ready	Marks ready WHEN rider arrives	Marks ready BEFORE food done
FOR Bias: 0 min	**FOR Bias: +positive**	**FOR Bias: −negative**
✓ Reliable signal	✗ Useless signal — too late	✗ Rider waits at restaurant

Dataset Reality: 40.1% Honest | 40.0% Rider-Triggered | 19.9% Early Marker → **59.9% of orders have biased FOR signals**

Business Impact of Biased FOR:

→ Early rider dispatch → idle waiting at restaurant

→ KPT model trained on noisy labels → poor predictions

→ Customer ETA errors → dissatisfaction & cancellations

CUARD SOLUTION ARCHITECTURE

1
FOR Signal Cleaning
Outlier clipping 1st–99th percentile

→

2
De-biased FOR Correction
Per merchant type correction logic

→

3
Merchant Reliability Score
0–1 trust index per merchant

→

4
Extended KCI
Zomato + non-Zomato kitchen load

→

5
Uncertainty Sigma
KCI × MRS adjusted dispatch window

6
IoT Buzzer Hardware
₹500 device eliminate bias at source

→

7
GBM Dispatch Policy
Gradient Boosting adaptive per order

→

8
SLA Constraint Layer
P(late > 3min) ≤ 10% Monte Carlo check

→

9
Dispatch Signal
Rider App <100ms latency

⭐ **Key Novelties:**

✓ De-biased FOR per merchant behavioral type

✓ Extended KCI with non-Zomato load (Google Maps API + BLE IoT)

✓ IoT Smart Buzzer — hardware signal capture

✓ SLA-probabilistic dispatch constraint

SIGNAL QUALITY: DE-BIASED FOR CORRECTION

Correction Logic by Merchant Type

honest

```
corrected_FOR = FOR_time
```

Trusted as-is, no correction needed

rider_triggered

```
corrected_FOR = order_time
+ hist_avg_prep(merchant)
```

Replace signal with historical average

early_marker

```
corrected_FOR = FOR_time
+ |FOR_bias_clean|
```

Shift forward by measured bias magnitude

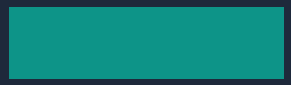
Signal Quality Improvement

FOR Signal MAE vs Actual Prep Time

Raw Signal:

9.71 min

Corrected:



4.63 min

52.3%

Signal Quality Improvement

IoT SMART BUZZER — ELIMINATING BIAS AT THE SOURCE

How It Works

1. Kitchen staff presses buzzer when packaging complete
2. ₹500 IoT device sends timestamped signal via WiFi/SIM
3. Signal reaches Zomato servers — bypasses Merchant App
4. BLE beacon passively counts footfall → feeds Extended KCI

Business Case

Device Cost	**₹500 / unit**
Phase 1 (30K merchants)	**₹2.1 Crore**
Annual Maintenance	**₹30 Lakh**
Conservative Savings	**₹90+ Crore/year**
Payback Period	**< 1 month**

59.9% of orders — FOR bias eliminated with IoT rollout

EXTENDED KCI: CAPTURING NON-ZOMATO KITCHEN LOAD

The Problem:
Zomato only sees its own orders. A restaurant may simultaneously handle dine-in + Swiggy — all invisible to KPT model.

KCI_extended = **0.6 × KCI_zomato** + **0.4 × non_zomato_multiplier**

Google Maps Popular Times API

Free hourly busyness per restaurant.
Fetch daily, merge as congestion prior.

POS Integration via Merchant App

Request transaction velocity via SDK.
Opt-in for premium merchants first.

WiFi/BLE Probe Count (IoT)

Device count ≈ customer footfall.
Same IoT buzzer hardware — no extra cost.

Result: KCI std improved from **0.083 (Zomato-only)** → **0.136 (Extended)** | +65% variance captured — real kitchen rush dynamics reflected

FINAL RESULTS — TIME-SPLIT VALIDATION (HELD-OUT FUTURE DATA)

Strategy	Avg Wait	P90 Wait	High Wait %	Improvement
Baseline (Current System)	~3.4 min	~6.7 min	~54%	—
Corrected Analytical (Z=0.75, KCI_ext)	~1.5 min	~3.2 min	~22%	+56%
GBM Policy + SLA Layer (CUARD Final)	~1.2 min	~2.8 min	~16%	+65%

65%
Rider Wait Reduction

52%
FOR Signal Improvement

₹90cr+
Annual Savings

<1mo
IoT Buzzer Payback

📌 **Validation Note:** All metrics computed on held-out FUTURE data (80/20 time-split). Conservative field estimate (20–30% of simulation improvement) still yields **₹25–50 crore/year** — a 15x return on IoT hardware investment.

SCALABILITY & PRODUCTION ARCHITECTURE

Order Confirmed › Kafka Event Bus › Flink Processor › GBM (ONNX) › SLA Check › Dispatch → Rider

Total Pipeline Latency: <50ms end-to-end (Flink: 20ms | Redis: 1ms | ONNX: 2ms | SLA: 10ms)

Cold-Start Handling

→ New merchant → city_tier median MRS

→ base_sigma = cuisine_category median

→ KCI = 1.0 (neutral until history builds)

→ Transitions to own params after 50 orders

Scale Numbers

→ 300K+ merchants supported

→ 1M+ daily orders processed

→ ~35 peak orders/second

→ No stateful joins — horizontal scale

Phased IoT Rollout

→ Phase 1: Top 10% high-volume (30K merchants)

→ Phase 2: Mid-tier merchants (100K)

→ Phase 3: Full merchant base (300K)

→ Each phase self-funded from savings

Fully scalable on commodity hardware — Redis + Flink + lightweight ONNX inference

Thank You

CUARD — Building a Smarter Dispatch System for Zomato Riders

52% MAE Reduction	65% Reduction	₹90+ Crore	59.9% Biased Orders Fixed
FOR Signal Quality	Rider Wait Time	Annual Savings	IoT Buzzer Coverage

Dataset: 45,584 delivery orders · Time-Split Validation · Production-Ready Architecture